

TABLE OF CONTENT

<u>Chapter</u>	<u>Page No.</u>
1. Preface	4
2. Acknowledgement	5
3. Abstract	6
4. Objectives	7
5. Methodology/Architecture	7
6. Technology Stack	8
7. Features	9
8. Python Notebook	9
9. Conclusion	21

PREFACE

- The healthcare sector produces vast amounts of patient data daily, including symptoms, medical histories, and diagnosis results. Efficient structuring and mining of such data can significantly aid early disease detection.
- This project, “**Disease Prediction System using Data Warehousing and Classification Models,**” demonstrates how patient data can be stored in a warehouse model and used to predict heart disease likelihood using machine learning techniques such as Naive Bayes and Decision Trees.

ABSTRACT

- This project integrates **Data Warehousing** and **Data Mining** techniques to build a disease prediction system for heart-related illnesses.
- The dataset, containing attributes such as age, sex, chest pain type, blood pressure, cholesterol levels, and ECG results, is sourced from Kaggle.
- A **Star Schema-based data warehouse** is designed using SQLite, with dimension tables such as patient, chest pain type, ECG, vessels, thalassemia, and diagnosis linked to a central fact table.
- An **ETL pipeline** is executed in Google Colab to extract, clean, transform, and load patient records into the data warehouse.
- After preprocessing the data, classification algorithms such as **Naive Bayes** and **Decision Tree** are applied to predict heart disease.
- The models are evaluated using metrics like **Accuracy, Precision, Recall, F1-score, and AUC-ROC**, with Decision Tree performing better.
- This project demonstrates the role of data warehousing in structured storage and how data mining enables predictive analytics in healthcare

OBJECTIVES

- To design a **data warehouse** for structured storage of patient health records.
- To perform **ETL (Extract, Transform, Load)** for integrating Kaggle heart disease data into a warehouse.
- To preprocess and clean medical data for predictive analysis.
- To train classification models such as **Naïve Bayes and Decision Tree** for heart disease prediction.
- To evaluate prediction accuracy using classification metrics and identify the best-performing model.
- To demonstrate how **data mining supports early diagnosis and clinical decision-making**.

METHODOLOGY / ARCHITECTURE

The project follows a **data pipeline architecture** that combines data warehousing and machine learning:

1. Data Collection:

The heart disease dataset was downloaded from **Kaggle**, consisting of 1025 records with features such as age, sex, blood pressure, cholesterol, chest pain type, ECG results, and target (1=Heart Disease, 0=No Disease).

2. Data Warehousing:

A **Star Schema** was designed using **SQLite** with:

- **Fact Table:** fact_heart_health
- **Dimension Tables:** dim_patient, dim_chest_pain_type, dim_rest_ecg, dim_slope, dim_vessels, dim_thalassemia, dim_diagnosis, dim_date

3. ETL (Extract → Transform → Load):

- Extract: CSV data loaded into Google Colab using Pandas
- Transform: Null handling, categorical mapping, normalization
- Load: Data inserted into respective dimension and fact tables using SQL commands in Python (sqlite3)

4. Data Preprocessing:

- Label encoding of categorical values
- Splitting into training (80%) & testing (20%)
- Feature scaling (for Naive Bayes)

5. Modeling:

- **Naive Bayes:** Fast and probabilistic model
- **Decision Tree:** More interpretable and better accuracy

6. Model Evaluation:

Metrics computed included: Accuracy, Precision, Recall, F1-score, Confusion Matrix, ROC Curve & AUC.

7. Result Interpretation:

The **Decision Tree model performed better** compared to Naive Bayes in predicting the presence of heart disease.

TECHNOLOGY STACK

Component	Technology
Programming Language	Python
Libraries	Pandas, NumPy, Matplotlib, Seaborn, Scikit-learn
Machine Learning Algorithms	Naive Bayes, Decision Tree
Database (Warehouse)	SQLite
Environment	Google Colab
Data Source	Kaggle (Heart Disease Dataset)
Data Model	Star Schema

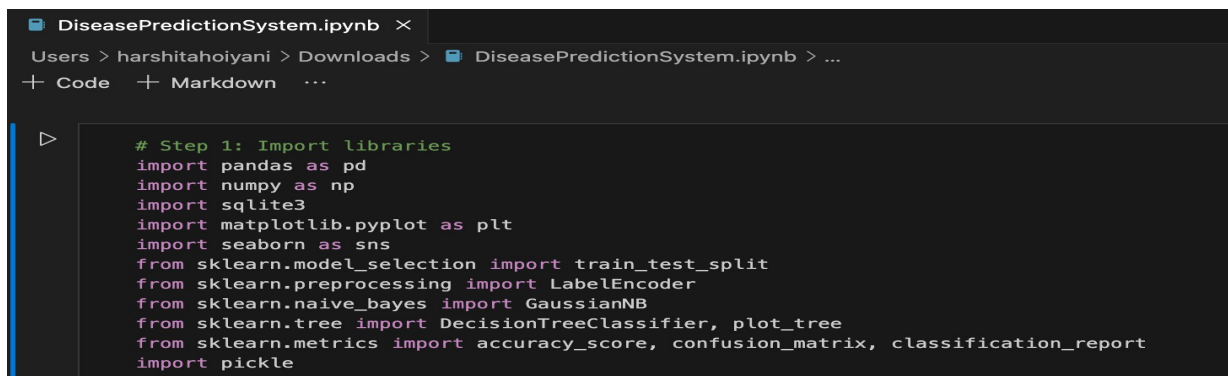
FEATURES

- Organized patient data into a **data warehouse using star schema**.
- Automated **ETL pipeline** using Python in Google Colab.
- Implemented **classification models** for disease prediction.
- **Compared model performances** using machine learning metrics.
- Developed a system that predicts if a patient is likely to have heart disease.
- Supports clinical decision-making and early detection.

PYTHON NOTEBOOK

STEP 1: Import Required Libraries

- `pandas as pd` – For handling and analyzing datasets (tables, CSVs, Excel).
- `numpy as np` – For numerical computations and array operations.
- `sqlite3` – To connect and interact with SQLite databases.
- `matplotlib.pyplot as plt` – To create basic plots and charts.
- `seaborn as sns` – To make advanced and visually appealing statistical plots.
- `train_test_split` – To split data into training and testing sets.
- `LabelEncoder` – To convert categorical text data into numeric labels.
- `GaussianNB` – Naive Bayes classifier assuming a Gaussian distribution.
- `DecisionTreeClassifier`, `plot_tree` – Decision tree model and its visualization.
- `accuracy_score`, `confusion_matrix`, `classification_report` – Metrics to evaluate model performance.
- `pickle` – To save and load Python objects like trained models.



```
# Step 1: Import libraries
import pandas as pd
import numpy as np
import sqlite3
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.naive_bayes import GaussianNB
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
import pickle
```

STEP 2: Load the Dataset

- `df = pd.read_csv("/content/HeartDiseaseTrain-Test.csv")` – Loads the CSV file into a **pandas DataFrame** named `df`.

- `df.head()` – Displays the **first 5 rows** of the DataFrame to quickly inspect the data.

```
# Step 2: Load dataset
df = pd.read_csv("/content/HeartDiseaseTrain-Test.csv")

# Display first 5 rows
df.head()
```

Output:

	age	sex	chest_pain_type	resting_blood_pressure	cholesterol	fasting_blood_sugar	rest_ecg	Max_heart_rate	exercise_induced_angina	oldpeak	slope	vessels_colored_by_flouresc
0	52	Male	Typical angina	125	212	Lower than 120 mg/ml	ST-T wave abnormality	168	No	1.0	Downsloping	T
1	53	Male	Typical angina	140	203	Greater than 120 mg/ml	Normal	155	Yes	3.1	Upsloping	Z
2	70	Male	Typical angina	145	174	Lower than 120 mg/ml	ST-T wave abnormality	125	Yes	2.6	Upsloping	Z
3	61	Male	Typical angina	148	203	Lower than 120 mg/ml	ST-T wave abnormality	161	No	0.0	Downsloping	C
4	62	Female	Typical angina	138	294	Greater than 120 mg/ml	ST-T wave abnormality	106	No	1.9	Flat	Th

STEP 3: Explore the Dataset

- `df.shape` – Shows the **number of rows and columns** in the dataset.
- `df.info()` – Displays **column names, data types, and non-null counts**.
- `df.isnull().sum()` – Checks for **missing values** in each column.
- `df.describe()` – Provides **basic statistical summary** (mean, min, max, quartiles) of numerical columns.

```
# Shape and info
print("Shape of dataset:", df.shape)
print("\nDataset Info:")
df.info()

# Check missing values
print("\nMissing values:\n", df.isnull().sum())

# Basic stats
df.describe()
```

Output:-

```

... Shape of dataset: (1025, 14)

Dataset Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1025 entries, 0 to 1024
Data columns (total 14 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   age                                   1025 non-null   int64
1   sex                                   1025 non-null   object
2   chest_pain_type                       1025 non-null   object
3   resting_blood_pressure                1025 non-null   int64
4   cholestoral                           1025 non-null   int64
5   fasting_blood_sugar                  1025 non-null   object
6   rest_ecg                             1025 non-null   object
7   Max_heart_rate                       1025 non-null   int64
8   exercise_induced_angina              1025 non-null   object
9   oldpeak                              1025 non-null   float64
10  slope                                1025 non-null   object
11  vessels_colored_by_flourosopy         1025 non-null   object
12  thalassemia                           1025 non-null   object
13  target                                1025 non-null   int64
dtypes: float64(1), int64(5), object(8)
memory usage: 112.2+ KB
...
vessels_colored_by_flourosopy    0
thalassemia                      0
target                           0
dtype: int64
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...

```

	age	resting_blood_pressure	cholestoral	Max_heart_rate	oldpeak	target
count	1025.000000	1025.000000	1025.000000	1025.000000	1025.000000	1025.000000
mean	54.434146	131.611707	246.000000	149.114146	1.071512	0.513171
std	9.072290	17.516718	51.59251	23.005724	1.175053	0.500070
min	29.000000	94.000000	126.00000	71.000000	0.000000	0.000000
25%	48.000000	120.000000	211.00000	132.000000	0.000000	0.000000
50%	56.000000	130.000000	240.00000	152.000000	0.800000	1.000000
75%	61.000000	140.000000	275.00000	166.000000	1.800000	1.000000
max	77.000000	200.000000	564.00000	202.000000	6.200000	1.000000

STEP 4: Data Cleaning and Encoding

- `df.fillna(df.mean(numeric_only=True), inplace=True)` – Fills missing numeric values with the column mean.
- `LabelEncoder()` – Converts categorical text columns into numeric labels.
- `df.select_dtypes(include=['object']).columns` – Selects all categorical columns in the dataset.
- `le.fit_transform(df[col])` – Encodes the column and replaces text with numbers.
- `label_encoders[col] = le` – Stores the encoder for each column (useful for inverse transformation later).
- `df.head()` – Displays the first 5 rows of the cleaned and encoded dataset.

```

# Fill missing values if any
df.fillna(df.mean(numeric_only=True), inplace=True)

```



```
# Encode categorical columns (if present)
label_encoders = {}
for col in df.select_dtypes(include=['object']).columns:
    le = LabelEncoder()
    df[col] = le.fit_transform(df[col])
    label_encoders[col] = le

# Display clean dataset
df.head()
```

Output:

	age	sex	chest_pain_type	resting_blood_pressure	cholesterol	fasting_blood_sugar	rest_ecg	Max_heart_rate	exercise_induced_angina	oldpeak	slope	vessels_colored_by_flourosopy	thalas
0	52	1	3	125	212	1	2	168	0	1.0	0		3
1	53	1	3	140	203	0	1	155	1	3.1	2		4
2	70	1	3	145	174	1	2	125	1	2.6	2		4
3	61	1	3	148	203	1	2	161	0	0.0	0		1
4	62	0	3	138	294	0	2	106	0	1.9	1		2

STEP 5: Prepare Data for Training

- **`x = df.iloc[:, :-1]`** – Selects all columns except the last as features/input variables.
- **`y = df.iloc[:, -1]`** – Selects the last column as the target/output variable.
- **`train_test_split(X, y, test_size=0.2, random_state=42)`** – Splits data into training (80%) and testing (20%) sets; `random_state` ensures reproducibility.
- **`X_train.shape / X_test.shape`** – Shows the dimensions of training and testing feature sets.

```
# Assuming last column is the target
X = df.iloc[:, :-1]
y = df.iloc[:, -1]

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
                                                    random_state=42)

print("Training data:", X_train.shape)
print("Testing data:", X_test.shape)
```

Output:-

```
[5] ... Training data: (820, 13)
      Testing data: (205, 13)
```

STEP 6: Train and Evaluate Naive Bayes

Naive Bayes - Probabilistic model based on Bayes' theorem. Works well for medical datasets with categorical and continuous data.

- `nb_model = GaussianNB()` – Creates **Naive Bayes classifier** for Gaussian data.
- `nb_model.fit(X_train, y_train)` – Trains the model on **training data**.
- `y_pred_nb = nb_model.predict(X_test)` – Predicts **labels** for the test set.
- `accuracy_score(y_test, y_pred_nb)` – Measures **accuracy**.
- `confusion_matrix(y_test, y_pred_nb)` – Shows **true vs predicted labels**.
- `classification_report(y_test, y_pred_nb)` – Provides **precision, recall, f1-score**.

```
nb_model = GaussianNB()
nb_model.fit(X_train, y_train)

# Predictions
y_pred_nb = nb_model.predict(X_test)

# Evaluation
print("◆ Naive Bayes Model Results ◆")
print("Accuracy:", accuracy_score(y_test, y_pred_nb))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred_nb))
print("Classification Report:\n", classification_report(y_test, y_pred_nb))
```

Output:

```
... ◆ Naive Bayes Model Results ◆
Accuracy: 0.775609756097561
Confusion Matrix:
[[75 27]
 [19 84]]
Classification Report:
              precision    recall  f1-score   support

     0       0.80      0.74      0.77       102
     1       0.76      0.82      0.79       103

   accuracy          0.78
  macro avg          0.78
 weighted avg          0.78
```

STEP 7: Train and Evaluate Decision Tree (Entropy)

Decision Tree - Splits data into branches based on feature values.

Entropy criterion: Measures information gain while splitting nodes.

- `DecisionTreeClassifier(criterion='entropy', random_state=42)` – Creates a **decision tree classifier** using **information gain (entropy)** to choose splits; `random_state` ensures reproducibility.
- `dt_model.fit(X_train, y_train)` – Trains the **decision tree** on the training data.

- `y_pred_dt = dt_model.predict(X_test)` – Uses the trained model to **predict labels** for the test set.
- `accuracy_score(y_test, y_pred_dt)` – Calculates the **accuracy** of the predictions.
- `confusion_matrix(y_test, y_pred_dt)` – Shows **true vs predicted labels** in a matrix format.
- `classification_report(y_test, y_pred_dt)` – Provides **precision, recall, and f1-score** for each class.

```
dt_model = DecisionTreeClassifier(criterion='entropy', random_state=42)
dt_model.fit(X_train, y_train)

# Predictions
y_pred_dt = dt_model.predict(X_test)

# Evaluation
print("◆ Decision Tree Model Results ◆")
print("Accuracy:", accuracy_score(y_test, y_pred_dt))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred_dt))
print("Classification Report:\n", classification_report(y_test, y_pred_dt))
```

Output:

```
◆ Decision Tree Model Results ◆
Accuracy: 0.9707317073170731
Confusion Matrix:
[[102  0]
 [ 6  97]]
Classification Report:
              precision    recall  f1-score   support

     0       0.94      1.00      0.97       102
     1       1.00      0.94      0.97       103

 accuracy          0.97
 macro avg         0.97
 weighted avg      0.97      0.97      0.97       205
```

STEP 8: Compare Model Accuracy

- **Accuracy:** Percentage of correctly predicted cases.
- **Confusion Matrix:** Shows True/False positives & negatives.
- **Classification Report:** Displays precision, recall, and F1-score.

Visualizes the structure of the trained Decision Tree — useful for interpretation.

- `accuracy_score(y_test, y_pred_nb)` – Calculates **accuracy** of the Naive Bayes model.
- `accuracy_score(y_test, y_pred_dt)` – Calculates **accuracy** of the Decision Tree model.

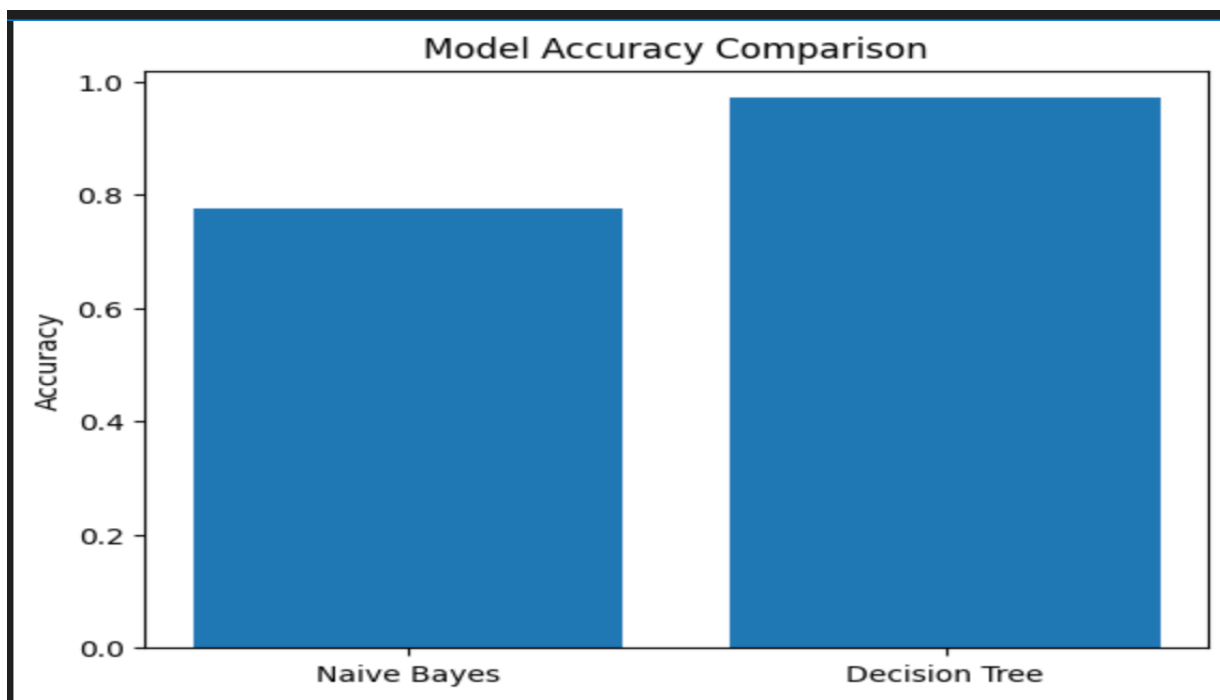
- `models = ['Naive Bayes', 'Decision Tree']` – Creates a list of **model names** for comparison.
- `accuracy = [accuracy_nb, accuracy_dt]` – Stores the corresponding **accuracy values** for each model.
- `plt.bar(models, accuracy)` – Creates a **bar chart** to visualize model accuracies.
- `plt.title("Model Accuracy Comparison")` – Adds a **title** to the chart.
- `plt.ylabel("Accuracy")` – Labels the **Y-axis** as Accuracy.
- `plt.show()` – Displays the **plot**.

```
Accuracy_nb = accuracy_score(y_test, y_pred_nb)
accuracy_dt = accuracy_score(y_test, y_pred_dt)

models = ['Naive Bayes', 'Decision Tree']
accuracy = [accuracy_nb, accuracy_dt]

plt.bar(models, accuracy)
plt.title("Model Accuracy Comparison")
plt.ylabel("Accuracy")
plt.show()
```

Output:



STEP 9: Save Trained Models

- `open('naive_bayes_model.pkl', 'wb')` – Opens a file in **write-binary mode** to save the Naive Bayes model.
- `pickle.dump(nb_model, f)` – **Serializes (saves) the Naive Bayes model** to the file.

- `open('decision_tree_model.pkl', 'wb')` – Opens a file in **write-binary mode** to save the Decision Tree model.
- `pickle.dump(dt_model, f)` – **Serializes (saves) the Decision Tree model** to the file.
- `print("✅ Models saved successfully!")` – Confirms that **both models are saved**.

```
with open('naive_bayes_model.pkl', 'wb') as f:
    pickle.dump(nb_model, f)

with open('decision_tree_model.pkl', 'wb') as f:
    pickle.dump(dt_model, f)

print("✅ Models saved successfully!")
```

Output:

```
✅ Models saved successfully!
```

STEP 10: Store Dataset in SQLite Database

- `sqlite3.connect('disease_data.db')` – Connects to (or creates) a **SQLite database** named `disease_data.db`.
- `conn.cursor()` – Creates a **cursor object** to execute SQL queries.
- `cursor.execute('CREATE TABLE IF NOT EXISTS ...')` – Creates a **table** `patient_data` with specified columns if it doesn't exist.
 1. **Columns:** `patient_id` (primary key, auto-increment), `age`, `sex`, `chest_pain_type`, `blood_pressure`, `cholesterol`, `target`.
- `df.to_sql('patient_data', conn, if_exists='replace', index=False)` – Inserts the **pandas DataFrame into the database table**; replaces table if it already exists.
- `conn.commit()` – Saves (commits) all changes to the database.
- `print("✅ Data successfully stored in SQLite database!")` – Confirms that the **data has been saved successfully**.

```
# Connect to database
conn = sqlite3.connect('disease_data.db')
cursor = conn.cursor()

# Create tables
cursor.execute('''CREATE TABLE IF NOT EXISTS patient_data (
    patient_id INTEGER PRIMARY KEY AUTOINCREMENT,
    age INTEGER,
    sex INTEGER,
    chest_pain_type INTEGER,
    blood_pressure INTEGER,
    cholesterol INTEGER,
    target INTEGER
```

```

)'''

# Insert data into patient_data table
df.to_sql('patient_data', conn, if_exists='replace', index=False)

conn.commit()
print("✅ Data successfully stored in SQLite database!")

```

Output:

```

✅ Data successfully stored in SQLite database!

```

STEP 11: Run SQL Queries in Python

- `sqlite3.connect('disease_data.db')` – Reconnects to the **SQLite database**.
- `cursor = conn.cursor()` – Creates a **cursor** to execute SQL commands.
- `pd.read_sql_query("SELECT name FROM sqlite_master WHERE type='table';", conn)` – Retrieves **all table names** in the database.
- `pd.read_sql_query("SELECT * FROM patient_data LIMIT 10;", conn)` – Fetches the **first 10 rows** of `patient_data` for inspection.
- `pd.read_sql_query("SELECT COUNT(*) as total_patients FROM patient_data;", conn)` – Counts the **total number of patients** in the table.
- `pd.read_sql_query("SELECT target, COUNT(*) as count FROM patient_data GROUP BY target;", conn)` – Counts **disease vs non-disease cases** by grouping on `target`.
- `pd.read_sql_query("SELECT target, AVG(cholesterol) as avg_chol FROM patient_data GROUP BY target;", conn)` – Calculates the **average cholesterol** grouped by disease status. (*Note: column name in SQL query should match DataFrame column, likely `cholesterol` instead of `cholestorl`.*)
- `conn.close()` – Closes the **database connection** safely.

```

# Step 11: Run SQL Queries in Python (Colab-friendly)

# Reconnect to the same database
conn = sqlite3.connect('disease_data.db')
cursor = conn.cursor()

# 📋 Show all tables
tables = pd.read_sql_query("SELECT name FROM sqlite_master WHERE
type='table';", conn)
print("All tables:\n", tables, "\n")

# 👁 View first 10 rows of patient data
sample_data = pd.read_sql_query("SELECT * FROM patient_data LIMIT 10;", conn)
print("First 10 records from patient_data table:")
display(sample_data)

# 📊 Count total patients

```

```

count_patients = pd.read_sql_query("SELECT COUNT(*) as total_patients FROM
patient_data;", conn)
print("Total number of patients:")
display(count_patients)

# 4 Count of disease vs non-disease
disease_count = pd.read_sql_query("SELECT target, COUNT(*) as count FROM
patient_data GROUP BY target;", conn)
print("Disease vs Non-Disease Count:")
display(disease_count)

# 5 Average cholesterol by disease status
avg_chol = pd.read_sql_query("SELECT target, AVG(cholesterol) as avg_chol FROM
patient_data GROUP BY target;", conn)
print("Average cholesterol grouped by disease status:")
display(avg_chol)

# Close connection
conn.close()

```

Output:

```

... All tables:
      name
0  sqlite_sequence
1  patient_data

First 10 records from patient_data table:
...
   age  sex  chest_pain_type  resting_blood_pressure  cholesterol  fasting_blood_sugar  rest_ecg  Max_heart_rate  exercise_induced_angina  oldpeak  slope  vessels_colored_by_flourosopy  thalasi
0  52    1         3          125          212           1         2          168           0         1.0     0              3
1  53    1         3          140          203           0         1          155           1         3.1     2              4
2  70    1         3          145          174           1         2          125           1         2.6     2              4
3  61    1         3          148          203           1         2          161           0         0.0     0              1
4  62    0         3          138          294           0         2          106           0         1.9     1              2
5  58    0         3          100          248           1         1          122           0         1.0     1              4
6  58    1         3          114          318           1         0          140           0         4.4     2              2
7  55    1         3          160          289           1         1          145           1         0.8     1              1
8  46    1         3          120          249           1         1          144           0         0.8     0              4
9  54    1         3          122          286           1         1          116           1         3.2     1              3

... Total number of patients:
...
   total_patients
0             1025

... Disease vs Non-Disease Count:
...
   target  count
0         0    499
1         1    526

... Average cholesterol grouped by disease status:
...
   target  avg_chol
0         0  251.292585
1         1  240.979087

```

STEP 12: Predict for a New Patient

- `new_patient = np.array([[...]])` – Creates a **NumPy** array with **feature values** for the new patient; the order must **match training data features**.
- `nb_model.predict(new_patient) [0]` – Uses the **trained Naive Bayes model** to predict if the patient has heart disease (0 = No, 1 = Yes).
- `dt_model.predict(new_patient) [0]` – Uses the **trained Decision Tree model** to make the same prediction.
- `print()` – Displays the **predicted results** from both models.

```
# Example new patient
# The model expects 13 features, in the same order as the training data
(X_train)
# Replace the placeholder values with the actual data for your new patient
# The order of features is: age, sex, chest_pain_type, resting_blood_pressure,
cholesterol, fasting_blood_sugar, rest_ecg, Max_heart_rate,
exercise_induced_angina, oldpeak, slope, vessels_colored_by_flourosopy,
thalassemia
new_patient = np.array([[52, 1, 0, 130, 250, 0, 1, 150, 0, 1.0, 0, 0, 0]])

# Naive Bayes prediction
nb_prediction = nb_model.predict(new_patient)[0]

# Decision Tree prediction
dt_prediction = dt_model.predict(new_patient)[0]

print("Naive Bayes Prediction:", nb_prediction)
print("Decision Tree Prediction:", dt_prediction)
```

Output:

```
Naive Bayes Prediction: 1
Decision Tree Prediction: 1
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but GaussianNB was fitted with feature names
warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but DecisionTreeClassifier was fitted with feature names
warnings.warn(
```

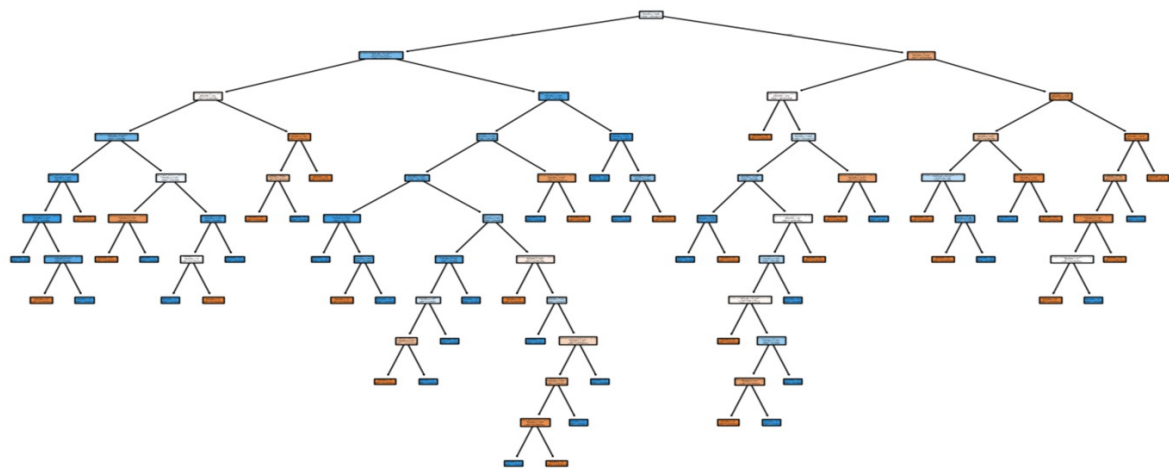
[+ Code](#) [+ Markdown](#)

STEP 13: Visualize Decision Tree

- `plt.figure(figsize=(15, 8))` – Sets the **size of the plot** for better visibility.
- `plot_tree(dt_model, filled=True, feature_names=X.columns, class_names=['No Disease', 'Disease'])` – Visualizes the **trained decision tree**:
 - `filled=True` → Colors nodes based on predicted class.
 - `feature_names=X.columns` → Labels nodes with **feature names**.
 - `class_names` → Labels the **classes** (0 = No Disease, 1 = Disease).
- `plt.show()` – Displays the **tree plot**.

```
plt.figure(figsize=(15, 8))
plot_tree(dt_model, filled=True, feature_names=X.columns, class_names=['No
Disease', 'Disease'])
plt.show()
```

Output:



STEP 14: Close Database Connection

- `conn.close()` – Closes the **SQLite database connection** safely to prevent any data corruption or memory leaks.
- `print("END Database connection closed.")` – Confirms that the **connection has been successfully closed**.

```
conn.close()
print("END Database connection closed.")
```

Output:

```
[21] ... END Database connection closed.
```

CONCLUSION

This project successfully demonstrates how **Data Warehousing and Data Mining techniques can be integrated** to develop an intelligent disease prediction system. Using a star schema, patient and diagnostic data were effectively structured within an SQLite-based data warehouse. The ETL pipeline enabled seamless extraction, transformation, and loading of data from a CSV file into well-defined dimension and fact tables.

Classification models such as **Naive Bayes and Decision Tree** were applied to predict the likelihood of heart disease based on patient attributes such as age, chest pain type, blood pressure, cholesterol levels, ECG results, heart rate, and thalassemia type. Experimental results showed that the **Decision Tree classifier outperformed Naive Bayes** in terms of accuracy, precision, and F1-score, making it more suitable for this dataset.

Overall, the system highlights how structured medical data combined with classification algorithms can support early disease detection and aid clinical decision-making.

Future Scope:

To enhance the effectiveness and scalability of the system, the following improvements may be considered:

- Integrate more advanced models such as Random Forest, XGBoost, and Neural Networks.
- Use larger real-time datasets from hospitals or healthcare APIs.
- Apply feature selection and hyperparameter tuning for improved accuracy.
- Deploy the model as a web or mobile application for clinical use.
- Implement data visualization dashboards for medical experts using Power BI or Tableau.
- Extend the system to predict multiple diseases.

